



UNIVERSITÉ MOHAMMED PREMIER – OUJDA  
FACULTÉ DES SCIENCES  
Département d'Informatique

Master M2I – Cloud Computing – Semestre 2

---

# Mini-Projet Cloud Computing

Projet 2 : Planification & Logistique

Microservices : Catalogue Cours – Locaux – Emploi du Temps

---

**Réalisé par :**

Genin Amine

El Kharraf Ayoub

Bensaadoun Aymane

**Encadré par :**

Pr. Abdelhak LAKHOUAJA

Année universitaire 2025 – 2026

# Remerciements

Nous tenons à exprimer notre sincère gratitude à notre encadrant, Pr.Abdelhak LA-KHOUAJA, pour son accompagnement, ses conseils avisés et sa disponibilité tout au long de la réalisation de ce mini-projet. Nous remercions également le Département d'Informatique de la Faculté des Sciences d'Oujda pour les moyens mis à notre disposition.

# Table des matières

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>Introduction générale</b>                        | <b>6</b>  |
| <b>1 Étude et conception</b>                        | <b>7</b>  |
| 1.1 Cahier des charges . . . . .                    | 7         |
| 1.2 Architecture générale du pôle . . . . .         | 7         |
| 1.3 Modèle de données . . . . .                     | 7         |
| <b>2 Phase 1 – Services individuels</b>             | <b>9</b>  |
| 2.1 Architecture commune . . . . .                  | 9         |
| 2.2 Service Catalogue Cours . . . . .               | 9         |
| 2.2.1 Modèle de données . . . . .                   | 9         |
| 2.2.2 API REST exposée . . . . .                    | 9         |
| 2.2.3 Dockerfile multi-stage . . . . .              | 9         |
| 2.3 Service Locaux . . . . .                        | 11        |
| 2.3.1 Modèle de données . . . . .                   | 11        |
| 2.3.2 API REST exposée . . . . .                    | 11        |
| 2.3.3 Dockerfile multi-stage . . . . .              | 11        |
| 2.4 Service Emploi du Temps . . . . .               | 12        |
| 2.4.1 Modèle de données . . . . .                   | 12        |
| 2.4.2 API REST exposée . . . . .                    | 13        |
| 2.4.3 Dockerfile multi-stage . . . . .              | 13        |
| <b>3 Phase 2 – Regroupement via Docker Compose</b>  | <b>15</b> |
| 3.1 Réseau Docker commun . . . . .                  | 15        |
| 3.2 Gestion de l'ordre de démarrage . . . . .       | 15        |
| 3.3 Fichier compose.yaml maître . . . . .           | 15        |
| 3.3.1 Vérification du déploiement . . . . .         | 16        |
| <b>4 Phase 3 – Infrastructure as Code (Vagrant)</b> | <b>18</b> |
| 4.1 Objectif . . . . .                              | 18        |
| 4.2 Vagrantfile . . . . .                           | 18        |
| 4.3 Provisioning . . . . .                          | 20        |
| 4.3.1 Démarrage de la machine virtuelle . . . . .   | 21        |
| 4.4 Vérification des machines virtuelles . . . . .  | 21        |
| <b>5 Phase 4 – Orchestration Kubernetes / K3s</b>   | <b>22</b> |
| 5.1 Installation du cluster K3s . . . . .           | 22        |
| 5.2 Vérification du cluster Kubernetes . . . . .    | 22        |
| 5.3 Sécurité . . . . .                              | 23        |

---

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| 5.3.1    | Secrets . . . . .                                       | 23        |
| 5.3.2    | ConfigMaps . . . . .                                    | 23        |
| 5.4      | Persistance des données . . . . .                       | 24        |
| 5.5      | Exposition via Ingress Controller . . . . .             | 25        |
| <b>6</b> | <b>Tests et validation</b>                              | <b>27</b> |
| 6.1      | Tests fonctionnels . . . . .                            | 27        |
| 6.1.1    | Validation des API REST depuis le navigateur . . . . .  | 27        |
| 6.1.2    | Validation des API REST avec la commande curl . . . . . | 28        |
| 6.2      | Difficultés rencontrées et solutions . . . . .          | 29        |
|          | <b>Conclusion générale</b>                              | <b>30</b> |
|          | <b>Bibliographie</b>                                    | <b>31</b> |

# Table des figures

|     |                                                                                                        |    |
|-----|--------------------------------------------------------------------------------------------------------|----|
| 1.1 | Architecture globale du pôle Planification & Logistique . . . . .                                      | 8  |
| 3.1 | Vérification des conteneurs avec la commande <code>docker compose ps</code> . . . . .                  | 17 |
| 4.1 | Machine virtuelle <code>k3s-master</code> exécutée dans Oracle VirtualBox . . . . .                    | 21 |
| 4.2 | Vérification de l'état des machines virtuelles avec la commande <code>vagrant status</code> . . . . .  | 21 |
| 5.1 | Vérification de l'état du cluster Kubernetes avec la commande <code>kubectl get nodes</code> . . . . . | 22 |
| 5.2 | Vérification des Pods déployés dans le namespace <code>planification</code> . . . . .                  | 23 |
| 5.3 | Secrets et ConfigMaps déployés dans le namespace <code>planification</code> . . . . .                  | 24 |
| 5.4 | StatefulSets et Persistent Volume Claims (PVC) . . . . .                                               | 25 |
| 5.5 | Configuration de l'Ingress Kubernetes . . . . .                                                        | 26 |
| 6.1 | Réponse JSON de l'API Catalogue affichée dans le navigateur . . . . .                                  | 28 |
| 6.2 | Test de l'API Catalogue avec la commande <code>curl</code> . . . . .                                   | 28 |

# Liste des tableaux

|     |                                                     |    |
|-----|-----------------------------------------------------|----|
| 2.1 | Modèle de données du service Catalogue . . . . .    | 9  |
| 2.2 | Endpoints du service Catalogue Cours . . . . .      | 10 |
| 2.3 | Endpoints du service Locaux . . . . .               | 11 |
| 2.4 | Endpoints REST du service Emploi du Temps . . . . . | 13 |

# Introduction générale

Le Cloud Computing constitue aujourd’hui un élément essentiel de la transformation numérique des systèmes d’information. En mettant à disposition des ressources informatiques à la demande, il offre une meilleure flexibilité, une grande capacité d’évolution ainsi qu’une simplification du déploiement et de l’administration des applications. Dans ce contexte, les architectures basées sur les microservices se sont largement imposées comme une solution adaptée au développement d’applications modernes, grâce à leur modularité, leur facilité de maintenance et leur capacité à être déployées indépendamment.

Dans le cadre du module **Cloud Computing** du Master **M2I** à la Faculté des Sciences d’Oujda, ce mini-projet a pour objectif de mettre en pratique les principaux concepts étudiés durant la formation. Le travail réalisé porte sur le développement du pôle **Planification & Logistique** d’un système universitaire, composé de plusieurs microservices collaborant pour assurer la gestion des cours, des locaux et des emplois du temps.

Au-delà du développement des fonctionnalités métier, ce projet met l’accent sur les différentes étapes du cycle de vie d’une application cloud. Il couvre notamment la conception d’une architecture distribuée, la conteneurisation des services, leur orchestration dans un environnement virtualisé ainsi que leur déploiement sur une plateforme Kubernetes. Cette démarche permet d’acquérir une expérience concrète des technologies et des pratiques utilisées dans les environnements cloud modernes.

Le présent rapport décrit l’ensemble des travaux réalisés. Il débute par une étude du projet et de son architecture, puis présente le développement des différents microservices. Les chapitres suivants sont consacrés à la conteneurisation avec Docker, à l’automatisation de l’infrastructure avec Vagrant, au déploiement sur Kubernetes/K3s, ainsi qu’aux tests effectués pour valider le bon fonctionnement de la solution proposée.

# Chapitre 1

## Étude et conception

### 1.1 Cahier des charges

L’Université Mohammed Premier (UMP) d’Oujda doit planifier chaque semestre des centaines de séances académiques (cours magistraux, TD, TP) en répartissant les modules dans des salles disponibles, sans conflit horaire. Le système à concevoir doit répondre aux exigences suivantes :

- **Gestion du catalogue des cours** : opérations CRUD sur les modules (code, intitulé, syllabus, prérequis, crédits ECTS).
- **Gestion des locaux** : inventaire des amphithéâtres, salles de TP et salles de cours, avec capacité, équipements et état de disponibilité (`DISPONIBLE`, `OCCUPE`, `EN_MAINTENANCE`).
- **Planification des séances** : création d’un emploi du temps en vérifiant automatiquement l’existence du cours, la disponibilité de la salle et l’absence de chevauchement horaire sur un même local.

L’architecture retenue repose sur trois microservices Jakarta EE 10 [6] indépendants, chacun avec sa propre base MySQL 8 [12] (*database per service*, communiquant par API REST HTTP. Le déploiement couvre quatre phases : développement unitaire, orchestration Docker Compose, virtualisation Vagrant et orchestration Kubernetes/K3s.

### 1.2 Architecture générale du pôle

- **Service Catalogue Cours** : gestion des modules, syllabus, prérequis.
- **Service Locaux** : gestion des amphithéâtres, salles de TP, capacités, équipements.
- **Service Emploi du Temps** : planification des séances, cœur du pôle.

### 1.3 Modèle de données

Chaque microservice possède sa propre base MySQL et son propre modèle JPA [7], sans clé étrangère inter-bases. Le couplage entre domaines s’effectue par identifiants logiques (`coursId`, `localId`) vérifiés via REST.



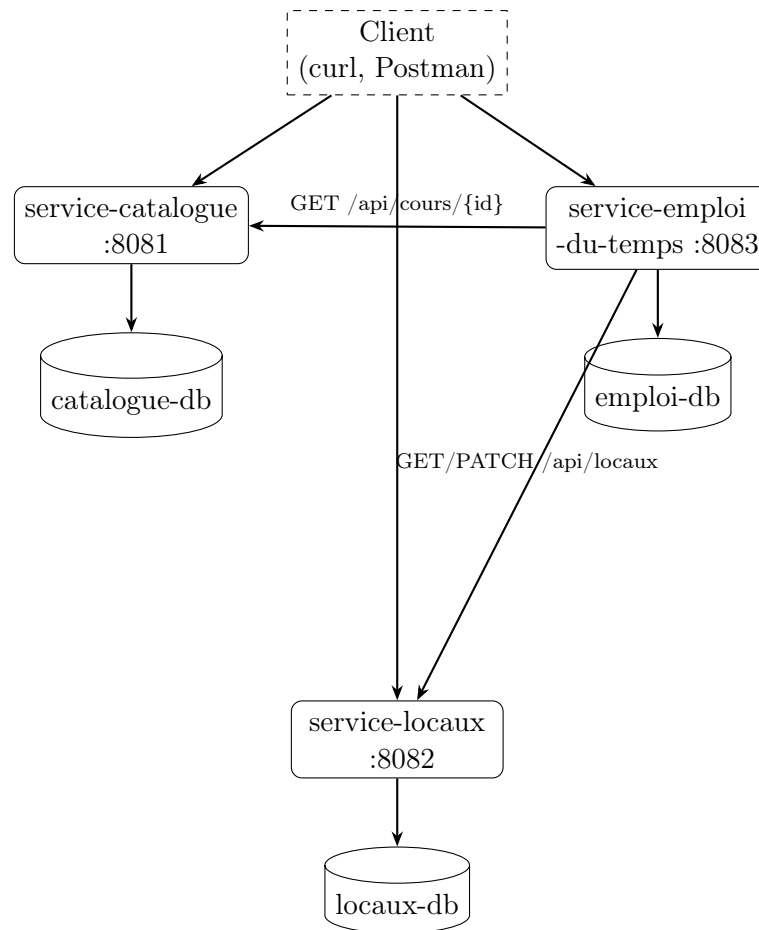
Réseau Docker `planification-net`

FIGURE 1.1 – Architecture globale du pôle Planification &amp; Logistique

**Service Catalogue Cours.** Entité Cours (table `cours`) : `id` (PK auto-incrémentée), `code` (unique, 20 caractères max), `intitule`, `syllabus` (TEXT), `prerequis`, `credits`. Dix cours pré-chargés via `init.sql`.

**Service Locaux.** Entité Local (table `locaux`) : `id`, `code` (unique), `nom`, `type` (enum : AMPHI, SALLE\_TP, SALLE\_COURS, LABO), `capacite`, `batiment`, `etage`, `équipements` (projecteur, tableauNumerique, climatisation, accessiblePMR), `disponibilite` (enum : DISPONIBLE, OCCUPE, EN\_MAINTENANCE).

**Service Emploi du Temps.** Entité Seance (table `seances`) : `id`, `coursId`, `localId`, `jour` (enum JourSemaine), `heureDebut` et `heureFin` (format HH:mm), `semestre`, `type` (enum COURS\_MAGISTRAL, TP, TD). Schéma généré par Hibernate [4] (`schema-generation.database.action=update`).

# Chapitre 2

## Phase 1 – Services individuels

### 2.1 Architecture commune

Chaque service repose sur deux conteneurs : un conteneur applicatif Jakarta EE déployé sur WildFly [16] exposant des API REST [8], et un conteneur de base de données MySQL [12].

### 2.2 Service Catalogue Cours

#### 2.2.1 Modèle de données

Le microservice **Catalogue** est responsable de la gestion des cours proposés par l’université. Chaque cours est enregistré dans une base de données MySQL et est identifié de manière unique par un identifiant numérique. Les principales informations stockées sont présentées dans le Tableau 2.1.

TABLE 2.1 – Modèle de données du service Catalogue

| Attribut  | Type         | Description                                  |
|-----------|--------------|----------------------------------------------|
| id        | INT          | Identifiant unique du cours.                 |
| code      | VARCHAR(20)  | Code du cours (ex. INF301).                  |
| intitule  | VARCHAR(255) | Intitulé du cours.                           |
| credits   | INT          | Nombre de crédits attribués au cours.        |
| prerequis | VARCHAR(255) | Liste des prérequis éventuels.               |
| syllabus  | TEXT         | Description du contenu pédagogique du cours. |

Ce modèle de données permet d’assurer la gestion des informations essentielles relatives aux cours tout en restant simple et facilement extensible.

#### 2.2.2 API REST exposée

#### 2.2.3 Dockerfile multi-stage

Chaque service utilise un **Dockerfile multi-stage** [11] : un premier stage Maven compile l’application, un second stage WildFly ne conserve que l’archive `.war` produite, ce qui réduit la taille finale de l’image.

| Méthode | Endpoint               | Description                                    |
|---------|------------------------|------------------------------------------------|
| GET     | /api/cours             | Liste tous les cours (HTTP 200, tableau JSON)  |
| GET     | /api/cours/{id}        | Retourne un cours par identifiant (200 ou 404) |
| GET     | /api/cours/code/{code} | Retourne un cours par code unique (200 ou 404) |
| POST    | /api/cours             | Crée un cours (201, corps JSON Cours)          |
| PUT     | /api/cours/{id}        | Met à jour un cours existant (200, 400 ou 404) |
| DELETE  | /api/cours/{id}        | Supprime un cours (204 ou 404)                 |

TABLE 2.2 – Endpoints du service Catalogue Cours

```

1  # Stage 1 -- BUILD
2  FROM maven:3.9-eclipse-temurin-17 AS build
3  WORKDIR /app
4  COPY pom.xml .
5  RUN mvn dependency:go-offline
6  COPY src ./src
7  RUN mvn clean package -DskipTests
8
9  # Stage 2 -- RUN
10 FROM quay.io/wildfly/wildfly:27.0.1.Final-jdk17
11
12 USER root
13 RUN mkdir -p /opt/jboss/wildfly/modules/com/mysql/main
14 COPY --from=build /root/.m2/repository/com/mysql/mysql-connector-j
15   /8.3.0/mysql-connector-j-8.3.0.jar \
16   /opt/jboss/wildfly/modules/com/mysql/main/mysql-connector-j.jar
17 COPY docker/module.xml /opt/jboss/wildfly/modules/com/mysql/main/
18   module.xml
19 COPY docker/entrypoint.sh /opt/jboss/entrypoint.sh
20 RUN sed -i 's/\r$//' /opt/jboss/entrypoint.sh \
21   && chmod +x /opt/jboss/entrypoint.sh \
22   && chown -R jboss:jboss /opt/jboss/wildfly/modules/com /opt/jboss/
23   entrypoint.sh
24
25 USER jboss
26 COPY --from=build /app/target/service-catalogue.war $JBOSS_HOME/
27   standalone/deployments/
28
29 ENV DATASOURCE_NAME=CatalogueDS
30 EXPOSE 8080
31
32 ENTRYPOINT ["/bin/bash", "/opt/jboss/entrypoint.sh"]

```

Listing 2.1 – Dockerfile – Catalogue Cours

## 2.3 Service Locaux

### 2.3.1 Modèle de données

Le service Locaux permet de gérer l'ensemble des espaces utilisés pour la planification des séances. Chaque local est représenté par une entité **Local**, stockée dans la table `locaux`. Cette entité contient les informations nécessaires pour identifier le local, connaître sa capacité, ses équipements et son état de disponibilité.

Les principaux attributs de l'entité **Local** sont :

- **id** : identifiant unique du local, généré automatiquement ;
- **code** : code unique du local, par exemple A1, TP2 ;
- **nom** : nom ou description du local ;
- **type** : type du local, par exemple AMPHI, SALLE\_TP, SALLE\_COURS ou LABO ;
- **capacite** : nombre maximal d'étudiants que le local peut accueillir ;
- **batiment** : bâtiment où se trouve le local ;
- **etage** : étage du local ;
- **projecteur**, **tableauNumerique**, **climatisation**, **accessiblePMR** : équipements disponibles dans le local ;
- **disponibilite** : état du local, avec les valeurs DISPONIBLE, OCCUPE ou EN\_MAINTENANCE.

Ce modèle permet au service Emploi du Temps de vérifier si un local existe et s'il est disponible avant de planifier une séance.

### 2.3.2 API REST exposée

Le service Locaux expose une API REST permettant de consulter, créer, modifier et supprimer des locaux. Il fournit également des opérations utiles pour la planification, comme la recherche des locaux disponibles ou la modification de l'état de disponibilité d'un local.

| Méthode | Endpoint                       | Description                                      |
|---------|--------------------------------|--------------------------------------------------|
| GET     | /api/locaux                    | Retourne la liste de tous les locaux.            |
| GET     | /api/locaux/{id}               | Retourne un local par son identifiant.           |
| GET     | /api/locaux/code/{code}        | Recherche un local par son code unique.          |
| GET     | /api/locaux/disponibles        | Retourne la liste des locaux disponibles.        |
| POST    | /api/locaux                    | Crée un nouveau local.                           |
| PUT     | /api/locaux/{id}               | Met à jour les informations d'un local existant. |
| PATCH   | /api/locaux/{id}/disponibilite | Modifie l'état de disponibilité d'un local.      |
| DELETE  | /api/locaux/{id}               | Supprime un local.                               |

TABLE 2.3 – Endpoints du service Locaux

### 2.3.3 Dockerfile multi-stage

```
1  # Stage 1 -- BUILD
2  FROM maven:3.9-eclipse-temurin-17 AS build
3  WORKDIR /app
4  COPY pom.xml .
5  RUN mvn dependency:go-offline
6  COPY src ./src
7  RUN mvn clean package -DskipTests
8
9  # Stage 2 -- RUN
10 FROM quay.io/wildfly/wildfly:27.0.1.Final-jdk17
11
12 USER root
13 RUN mkdir -p /opt/jboss/wildfly/modules/com/mysql/main
14 COPY --from=build /root/.m2/repository/com/mysql/mysql-connector-j
15   /8.3.0/mysql-connector-j-8.3.0.jar \
16   /opt/jboss/wildfly/modules/com/mysql/main/mysql-connector-j.jar
17 COPY docker/module.xml /opt/jboss/wildfly/modules/com/mysql/main/
18   module.xml
19 COPY docker/entrypoint.sh /opt/jboss/entrypoint.sh
20 RUN sed -i 's/\r$//' /opt/jboss/entrypoint.sh \
21   && chmod +x /opt/jboss/entrypoint.sh \
22   && chown -R jboss:jboss /opt/jboss/wildfly/modules/com /opt/jboss/
23   entrypoint.sh
24
25 USER jboss
26 COPY --from=build /app/target/service-locaux.war $JBOSS_HOME/
27   standalone/deployments/
28
29 ENV DATASOURCE_NAME=LocauxDS
30 EXPOSE 8080
31
32 ENTRYPOINT ["/bin/bash", "/opt/jboss/entrypoint.sh"]
```

Listing 2.2 – Dockerfile – Locaux

## 2.4 Service Emploi du Temps

### 2.4.1 Modèle de données

Le service **Emploi du Temps** est le microservice responsable de la planification des séances d'enseignement. Il centralise les informations relatives aux horaires tout en communiquant avec les services *Catalogue* et *Locaux* afin de vérifier l'existence des cours et la disponibilité des salles avant toute planification.

L'entité principale est **Seance**, stockée dans la table `seances`. Contrairement à une base de données monolithique, les références vers les cours et les locaux sont réalisées uniquement par leurs identifiants (`coursId` et `localId`), sans clé étrangère, chaque microservice possédant sa propre base de données.

Les principaux attributs de l'entité **Seance** sont :

- **id** : identifiant unique de la séance ;
- **coursId** : identifiant du cours provenant du service Catalogue ;
- **localId** : identifiant du local provenant du service Locaux ;
- **jour** : jour de la semaine (LUNDI à SAMEDI) ;

- **heureDebut** : heure de début de la séance ;
- **heureFin** : heure de fin de la séance ;
- **semestre** : semestre concerné ;
- **type** : type de séance (Cours Magistral, TD ou TP).

Avant l'enregistrement d'une nouvelle séance, le service vérifie automatiquement l'existence du cours, la disponibilité du local ainsi que l'absence de conflits d'horaires afin de garantir la cohérence de la planification.

## 2.4.2 API REST exposée

Le service Emploi du Temps expose une API REST permettant de créer, consulter et supprimer des séances d'enseignement. Lors de la création d'une séance, il interagit avec les services Catalogue et Locaux afin de valider les informations reçues et de réserver automatiquement le local sélectionné.

| Méthode | Endpoint                         | Description                                                                                          |
|---------|----------------------------------|------------------------------------------------------------------------------------------------------|
| GET     | /api/emploi-du-temps             | Retourne toutes les séances planifiées.                                                              |
| GET     | /api/emploi-du-temps/{id}        | Retourne une séance selon son identifiant.                                                           |
| GET     | /api/emploi-du-temps/jour/{jour} | Retourne les séances planifiées pour un jour donné.                                                  |
| POST    | /api/emploi-du-temps             | Planifie une nouvelle séance après validation du cours, du local et de l'absence de conflit horaire. |
| DELETE  | /api/emploi-du-temps/{id}        | Supprime une séance et libère automatiquement le local réservé.                                      |

TABLE 2.4 – Endpoints REST du service Emploi du Temps

Le service agit comme un orchestrateur des autres microservices. Lors d'une planification, il interroge le service Catalogue pour vérifier l'existence du cours, consulte le service Locaux pour s'assurer que la salle est disponible, puis met à jour son état en la marquant comme occupée. Lors de la suppression d'une séance, le local est automatiquement remis à l'état DISPONIBLE.

## 2.4.3 Dockerfile multi-stage

```

1  # Stage 1 -- BUILD
2  FROM maven:3.9-eclipse-temurin-17 AS build
3  WORKDIR /app
4  COPY pom.xml .
5  RUN mvn dependency:go-offline
6  COPY src ./src
7  RUN mvn clean package -DskipTests
8
9  # Stage 2 -- RUN
10 FROM quay.io/wildfly/wildfly:27.0.1.Final-jdk17
11
12 USER root
13 RUN mkdir -p /opt/jboss/wildfly/modules/com/mysql/main

```

```
14 COPY --from=build /root/.m2/repository/com/mysql/mysql-connector-j
    /8.3.0/mysql-connector-j-8.3.0.jar \
15 /opt/jboss/wildfly/modules/com/mysql/main/mysql-connector-j.jar
16 COPY docker/module.xml /opt/jboss/wildfly/modules/com/mysql/main/
    module.xml
17 COPY docker/entrypoint.sh /opt/jboss/entrypoint.sh
18 RUN sed -i 's/\r$//' /opt/jboss/entrypoint.sh \
19 && chmod +x /opt/jboss/entrypoint.sh \
20 && chown -R jboss:jboss /opt/jboss/wildfly/modules/com /opt/jboss/
    entrypoint.sh
21
22 USER jboss
23 COPY --from=build /app/target/service-emploi-du-temps.war
    $JBOSS_HOME/standalone/deployments/
24
25 ENV DATASOURCE_NAME=EmploiDS
26 ENV CATALOGUE_BASE_URL=http://service-catalogue:8080
27 ENV LOCAUX_BASE_URL=http://service-locaux:8080
28 EXPOSE 8080
29
30 ENTRYPOINT ["/bin/bash", "/opt/jboss/entrypoint.sh"]
```

Listing 2.3 – Dockerfile – Emploi du Temps

# Chapitre 3

## Phase 2 – Regroupement via Docker Compose

### 3.1 Réseau Docker commun

Les trois services communiquent via un réseau Docker interne, permettant les appels API REST Backend-to-Backend (B2B).

### 3.2 Gestion de l'ordre de démarrage

Le service Emploi du Temps étant central, sa disponibilité dépend de celle de Catalogue Cours et de Locaux. La directive `depends_on` étant insuffisante pour garantir la disponibilité effective d'une base de données, nous avons mis en place :

- Des **healthchecks MySQL** (`mysqladmin ping`) sur `catalogue-db`, `locaux-db` et `emploi-db`, avec `depends_on: condition: service_healthy` pour chaque application WildFly.
- Des **healthchecks HTTP** (`curl -f`) sur `/api/cours`, `/api/locaux` et `/api/emploi-du-temps`, avec `start_period: 60s` (WildFly démarre en 60–90 s).
- Dans le `compose.yaml` maître, `service-emploi-du-temps` attend en plus `service-catalogue` et `service-locaux` en état `healthy`, car l'orchestrateur les appelle dès le premier POST.
- Le binaire `wait-for-it` n'est pas utilisé ; l'attente est assurée par les healthchecks Compose, la fonction `wait_for_service()` de `test.sh` et la boucle `curl` de `vagrant/start-app.sh`.

### 3.3 Fichier `compose.yaml` maître

Le format et les directives utilisés dans les fichiers `compose.yaml` (services, réseaux, healthchecks, `depends_on`) suivent la référence officielle du format Compose [2].

```
1  # Projet 2 -- Phase 2 : Orchestration maître des microservices UMP
2  # Usage : docker compose up --build -d
3  # Réseau partagé : planification-net
4
5  include:
6  - path: ../service-catalogue/compose.yaml
```



```
7   - path: ./service-locaux/compose.yaml
8   - path: ./service-emploi-du-temps/compose.yaml
9
10  services:
11    service-catalogue:
12      healthcheck:
13        test: ["CMD", "curl", "-f", "http://localhost:8080/api/cours"]
14        interval: 15s
15        timeout: 5s
16        retries: 3
17        start_period: 60s
18
19    service-locaux:
20      healthcheck:
21        test: ["CMD", "curl", "-f", "http://localhost:8080/api/locaux"]
22        interval: 15s
23        timeout: 5s
24        retries: 3
25        start_period: 60s
26
27    service-emploi-du-temps:
28      depends_on:
29        emploi-db:
30          condition: service_healthy
31        service-catalogue:
32          condition: service_healthy
33        service-locaux:
34          condition: service_healthy
35      healthcheck:
36        test: ["CMD", "curl", "-f", "http://localhost:8080/api/emploi-du-temps"]
37        interval: 15s
38        timeout: 5s
39        retries: 3
40        start_period: 60s
41
42  networks:
43    planification-net:
44      name: planification-net
45      driver: bridge
```

Listing 3.1 – compose.yaml maître (directive include)

### 3.3.1 Vérification du déploiement

Après l'exécution de la commande `docker compose up -build -d`, Docker Compose construit les images des microservices, crée les conteneurs et démarre l'ensemble de l'application. Il est ensuite possible de vérifier l'état des services grâce à la commande `docker compose ps`.

Comme l'illustre la Figure 3.1, les trois microservices (`service-catalogue`, `service-locaux` et `service-emploi-du-temps`) ainsi que leurs bases de données MySQL sont correctement démarrés. L'état *healthy* indique que les tests de santé (*health checks*) définis dans le fichier `compose.yaml` ont été validés avec succès.

```
vagrant@k3s-master:~/projet2-planification$ docker compose ps
```

| NAME                                        | IMAGE                                         | COMMAND                  | SERVICE                 | CREATED        | STATUS                  |
|---------------------------------------------|-----------------------------------------------|--------------------------|-------------------------|----------------|-------------------------|
| catalogue-db                                | mysql:8.0                                     | "docker-entrypoint.s..." | catalogue-db            | 35 minutes ago | Up 35 minutes (healthy) |
| 3306/tcp, 33060/tcp                         |                                               |                          |                         |                |                         |
| emploi-db                                   | mysql:8.0                                     | "docker-entrypoint.s..." | emploi-db               | 35 minutes ago | Up 35 minutes (healthy) |
| 3306/tcp, 33060/tcp                         |                                               |                          |                         |                |                         |
| locaux-db                                   | mysql:8.0                                     | "docker-entrypoint.s..." | locaux-db               | 35 minutes ago | Up 35 minutes (healthy) |
| 3306/tcp, 33060/tcp                         |                                               |                          |                         |                |                         |
| service-catalogue                           | projet2-planification-service-catalogue       | "/bin/bash /opt/jbos..." | service-catalogue       | 35 minutes ago | Up 27 minutes (healthy) |
| 0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp |                                               |                          |                         |                |                         |
| service-emploi-du-temps                     | projet2-planification-service-emploi-du-temps | "/bin/bash /opt/jbos..." | service-emploi-du-temps | 35 minutes ago | Up 15 minutes (healthy) |
| 0.0.0.0:8083->8080/tcp, [::]:8083->8080/tcp |                                               |                          |                         |                |                         |
| service-locaux                              | projet2-planification-service-locaux          | "/bin/bash /opt/jbos..." | service-locaux          | 35 minutes ago | Up 27 minutes (healthy) |
| 0.0.0.0:8082->8080/tcp, [::]:8082->8080/tcp |                                               |                          |                         |                |                         |

```
vagrant@k3s-master:~/projet2-planification$
```

FIGURE 3.1 – Vérification des conteneurs avec la commande `docker compose ps`

# Chapitre 4

## Phase 3 – Infrastructure as Code (Vagrant)

### 4.1 Objectif

Scripter la création de l'infrastructure de production à l'aide d'un Vagrantfile [5].

### 4.2 Vagrantfile

```
1  # -*- mode: ruby -*-
2  # vi: set ft=ruby :
3  #
4  # Phase 3/4 -- Vagrant : cluster K3s (master + worker) ou Docker
   Compose
5  # Projet 2 -- Planification & Logistique -- UMP Oujda
6  #
7  # Prérequis hôte : VirtualBox + Vagrant
8  # Usage :
9  #   vagrant up k3s-master          # VM principale (Docker Compose
   ou K3s)
10 #   vagrant up k3s-worker          # Nœud worker K3s
11 #   vagrant up                     # Démarre les deux VMs
12 #
13 # Depuis Windows (ports forwardés sur k3s-master) :
14 #   Docker Compose : http://localhost:18081/api/cours (évite
   conflit avec Docker Desktop local)
15 #   K3s NodePort   : http://localhost:30081/api/cours
16 #   API K3s        : https://localhost:6443
17 # Si Docker Desktop n'est pas lancé sur l'hôte, vous pouvez remapper
   host: 8081-8083.
18
19 Vagrant.configure("2") do |config|
20
21   # Box commune aux deux VMs
22   config.vm.box = "ubuntu/jammy64"
23
24   # Premier boot Ubuntu + VirtualBox 7 sur Windows : SSH peut prendre
   > 5 min
25   config.vm.boot_timeout = 600
26   config.ssh.connect_timeout = 60
27   config.ssh.keep_alive = true
```

```
28
29 # Dossier du projet synchronisé (lecture depuis l'hôte Windows)
30 # Docker/K8s ne supportent pas bien vboxsf → copie native dans start
   -app.sh
31 config.vm.synced_folder ".", "/vagrant",
32 mount_options: ["dmode=777", "fmode=666"]
33
34 # -----
35 # k3s-master -- VM principale (192.168.56.10, 4 Go RAM)
36 # Phase 3 : Docker Compose | Phase 4 : K3s control-plane
37 # -----
38 config.vm.define "k3s-master", primary: true do |master|
39   master.vm.hostname = "k3s-master"
40   master.vm.network "private_network", ip: "192.168.56.10"
41
42   # Phase 3 -- Docker Compose (guest 8081-8083 → host 18081-18083 pour
   éviter conflit Docker Desktop)
43   master.vm.network "forwarded_port", guest: 8081, host: 18081,
   host_ip: "127.0.0.1", id: "catalogue"
44   master.vm.network "forwarded_port", guest: 8082, host: 18082,
   host_ip: "127.0.0.1", id: "locaux"
45   master.vm.network "forwarded_port", guest: 8083, host: 18083,
   host_ip: "127.0.0.1", id: "emploi"
46
47   # Phase 4 -- K3s NodePort + API server
48   master.vm.network "forwarded_port", guest: 30081, host: 30081,
   host_ip: "127.0.0.1", id: "k8s-catalogue", auto_correct: true
49   master.vm.network "forwarded_port", guest: 30082, host: 30082,
   host_ip: "127.0.0.1", id: "k8s-locaux", auto_correct: true
50   master.vm.network "forwarded_port", guest: 30083, host: 30083,
   host_ip: "127.0.0.1", id: "k8s-emploi", auto_correct: true
51   master.vm.network "forwarded_port", guest: 6443, host: 6443,
   host_ip: "127.0.0.1", id: "k8s-api", auto_correct: true
52
53   master.vm.provider "virtualbox" do |vb|
54     vb.name = "ump-planification-k3s-master"
55     vb.memory = 4096
56     vb.cpus = 2
57     # vb.gui = true # décommenter pour voir l'écran VM si boot bloqué
58     vb.customize ["modifyvm", :id, "--natdnshostresolver1", "on"]
59     vb.customize ["modifyvm", :id, "--ioapic", "on"]
60     # Correctifs courants VBox 7 + ubuntu/jammy64 (timeout SSH au boot)
61     vb.customize ["modifyvm", :id, "--uart1", "off"]
62     vb.customize ["modifyvm", :id, "--graphicscontroller", "vmsvga"]
63     vb.customize ["modifyvm", :id, "--audio", "none"]
64     vb.customize ["modifyvm", :id, "--clipboard", "disabled"]
65     vb.customize ["modifyvm", :id, "--draganddrop", "disabled"]
66   end
67
68   master.vm.provider "libvirt" do |lv|
69     lv.memory = 4096
70     lv.cpus = 2
71   end
72
73   # 1) Une seule fois : Docker, Git, curl, rsync
74   master.vm.provision "docker", type: "shell", path: "vagrant/
   provision.sh"
75
```

```

76 # 2) À chaque vagrant up : sync projet + Docker Compose OU mode K3s
77 master.vm.provision "app", type: "shell", path: "vagrant/start-app.
    sh", run: "always"
78 end
79
80 # -----
81 # k3s-worker -- Nœud worker (192.168.56.11, 2 Go RAM)
82 # Rejoint le cluster K3s via le master (installation manuelle Phase
    4)
83 # -----
84 config.vm.define "k3s-worker" do |worker|
85   worker.vm.hostname = "k3s-worker"
86   worker.vm.network "private_network", ip: "192.168.56.11"
87
88   worker.vm.provider "virtualbox" do |vb|
89     vb.name = "ump-planification-k3s-worker"
90     vb.memory = 2048
91     vb.cpus = 1
92     vb.customize ["modifyvm", :id, "--natdnshostresolver1", "on"]
93     vb.customize ["modifyvm", :id, "--ioapic", "on"]
94     vb.customize ["modifyvm", :id, "--uart1", "off"]
95     vb.customize ["modifyvm", :id, "--graphicscontroller", "vmsvga"]
96     vb.customize ["modifyvm", :id, "--audio", "none"]
97   end
98
99   worker.vm.provider "libvirt" do |lv|
100     lv.memory = 2048
101     lv.cpus = 1
102   end
103
104   # Docker requis pour l'agent K3s (k3s agent)
105   worker.vm.provision "docker", type: "shell", path: "vagrant/
    provision.sh"
106 end
107
108 end

```

Listing 4.1 – Vagrantfile

## 4.3 Provisioning

Le provisioning est entièrement réalisé en **shell bash** (pas d'Ansible). Deux VMs Ubuntu 22.04 (ubuntu/jammy64) sont définies :

- k3s-master (primaire) : IP 192.168.56.10, 4096 Mo RAM, 2 vCPU ; exécute vagrant/provision.sh (installation Docker Engine, Compose v2, git, curl, rsync) puis vagrant/start-app.sh à chaque vagrant up (rsync du projet vers /home/vagrant/projet2-planification, correction CRLF, docker compose up -build -d ou mode K3s).
- k3s-worker : IP 192.168.56.11, 2048 Mo RAM, 1 vCPU ; provisioning Docker uniquement, prêt pour rejoindre le cluster K3s manuellement.

Les ports hôte 18081-18083 (Compose) et 30081-30083 (NodePort K3s) sont forwardés depuis la VM master pour éviter les conflits avec Docker Desktop sur Windows.

### 4.3.1 Démarrage de la machine virtuelle

Après l'exécution de la commande `vagrant up`, la machine virtuelle `k3s-master` est créée automatiquement dans Oracle VirtualBox [1]. Cette machine exécute Ubuntu 22.04 et constitue le nœud maître du cluster Kubernetes. Elle héberge l'ensemble des composants nécessaires au déploiement de l'application, notamment Docker, Docker Compose et K3s.

La Figure 4.1 montre la machine virtuelle après son démarrage dans Oracle VirtualBox.

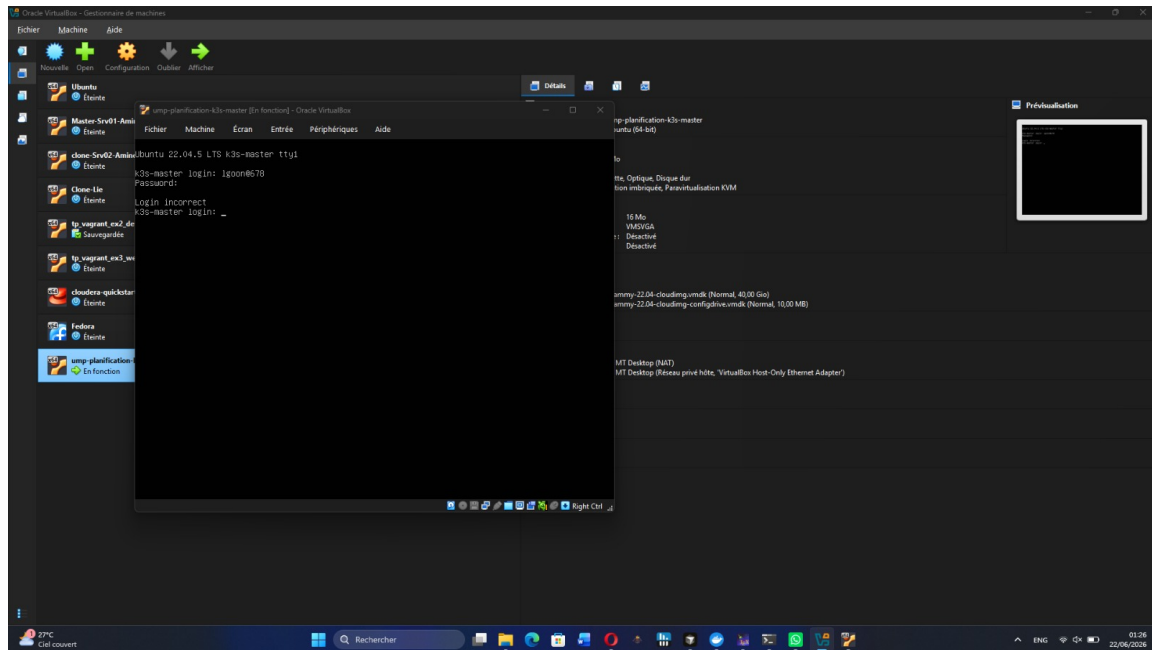


FIGURE 4.1 – Machine virtuelle `k3s-master` exécutée dans Oracle VirtualBox

## 4.4 Vérification des machines virtuelles

Après le provisioning et le démarrage des machines virtuelles, il est important de vérifier leur état avant de poursuivre le déploiement des conteneurs Docker et du cluster Kubernetes. Cette vérification est réalisée à l'aide de la commande `vagrant status`, qui affiche l'état de chaque machine définie dans le fichier `Vagrantfile`.

Dans notre projet, cette commande confirme que la machine virtuelle `k3s-master` est correctement démarrée (*running*) sous Oracle VirtualBox. Elle indique également que la machine `k3s-worker` n'a pas été créée (*not created*), ce qui est conforme à notre configuration de déploiement basée sur un seul nœud maître. Cette étape permet de s'assurer que l'environnement virtualisé est prêt avant le lancement des services Docker et des ressources Kubernetes.

```
PS C:\Users\hjp\Desktop\Waster_2025-2027\W2I-S2\cloud\projetFin\cloud-planification-microservices> vagrant status
Current machine states:

k3s-master      running (virtualbox)
k3s-worker      not created (virtualbox)

This environment represents multiple VMs. The VMs are all listed
above with their current state. For more information about a specific
VM, run `vagrant status NAME`.
PS C:\Users\hjp\Desktop\Waster_2025-2027\W2I-S2\cloud\projetFin\cloud-planification-microservices>
```

FIGURE 4.2 – Vérification de l'état des machines virtuelles avec la commande `vagrant status`

# Chapitre 5

## Phase 4 – Orchestration Kubernetes / K3s

### 5.1 Installation du cluster K3s

K3s [9] est installé sur la VM `k3s-master` (Ubuntu 22.04, 4 Go RAM) par la commande officielle `curl -sfL https://get.k3s.io | sh -`, produisant un cluster single-node avec Traefik (Ingress) et containerd. Le fichier `kubeconfig` est copié vers `~/.kube/config` pour l'utilisateur `vagrant`. Les images WildFly (`ump/service-*:1.0.0`) sont construites localement puis importées dans K3s via `k8s/scripts/import-images-k3s.sh`. Le déploiement complet s'effectue avec `kubectl apply -k k8s/ (namespace planification)` [10].

### 5.2 Vérification du cluster Kubernetes

Une fois l'installation de K3s terminée, il est indispensable de vérifier que le cluster Kubernetes est correctement initialisé et prêt à héberger les ressources de l'application. Cette vérification est réalisée à l'aide de la commande `kubectl get nodes`, qui affiche la liste des nœuds composant le cluster ainsi que leur état.

Dans notre projet, le cluster est constitué d'un nœud maître nommé `k3s-master`. Comme l'illustre la Figure 5.1, ce nœud apparaît avec l'état *Ready*, ce qui indique que le plan de contrôle (control plane) est opérationnel et que le cluster est capable de recevoir les différentes ressources Kubernetes, telles que les Pods, les Services, les ConfigMaps, les Secrets et les StatefulSets [10].

Cette étape constitue une validation essentielle avant le déploiement des microservices, car elle garantit que l'environnement Kubernetes est correctement configuré et fonctionnel.

```
vagrant@k3s-master:~/projet2-planification$ kubectl get nodes
NAME           STATUS    ROLES           AGE     VERSION
k3s-master     Ready     control-plane   5d18h   v1.35.5+k3s1
vagrant@k3s-master:~/projet2-planification$
```

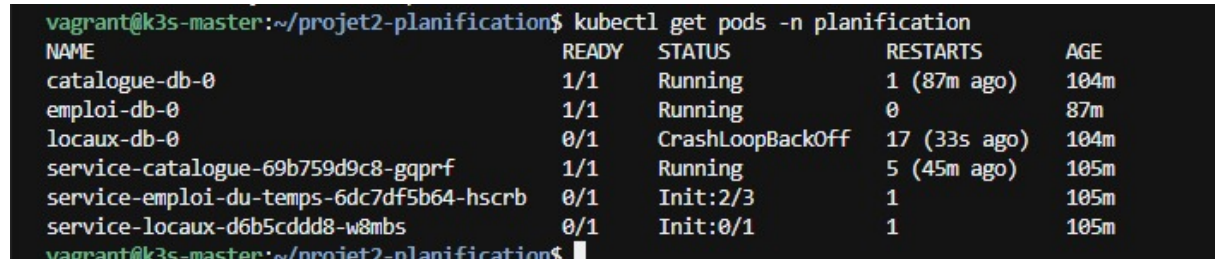
FIGURE 5.1 – Vérification de l'état du cluster Kubernetes avec la commande `kubectl get nodes`

Après la vérification des nœuds, il est également nécessaire de contrôler le déploiement des Pods correspondant aux différents composants de l'application. La commande `kubectl`



`get pods -n planification` affiche l'ensemble des Pods présents dans le namespace `planification` ainsi que leur état d'exécution.

La Figure 5.2 montre les Pods déployés pour les trois microservices ainsi que leurs bases de données. Cette vérification permet de confirmer que les ressources Kubernetes ont été créées correctement et que les services sont en cours de démarrage ou d'exécution.



```
vagrant@k3s-master:~/projet2-planification$ kubectl get pods -n planification
```

| NAME                                     | READY | STATUS           | RESTARTS     | AGE  |
|------------------------------------------|-------|------------------|--------------|------|
| catalogue-db-0                           | 1/1   | Running          | 1 (87m ago)  | 104m |
| emploi-db-0                              | 1/1   | Running          | 0            | 87m  |
| locaux-db-0                              | 0/1   | CrashLoopBackOff | 17 (33s ago) | 104m |
| service-catalogue-69b759d9c8-gqprf       | 1/1   | Running          | 5 (45m ago)  | 105m |
| service-emploi-du-temps-6dc7df5b64-hscrb | 0/1   | Init:2/3         | 1            | 105m |
| service-locaux-d6b5cddd8-w8mbs           | 0/1   | Init:0/1         | 1            | 105m |

FIGURE 5.2 – Vérification des Pods déployés dans le namespace `planification`

## 5.3 Sécurité

### 5.3.1 Secrets

Les informations sensibles (identifiants de connexion aux bases de données) sont stockées sous forme de ressources `Secret` [13], évitant ainsi de les inscrire en clair dans les spécifications des Pods ou dans les images de conteneurs.

```

1  apiVersion: v1
2  kind: Secret
3  metadata:
4    name: catalogue-db-secret
5    namespace: planification
6    labels:
7      app: catalogue-db
8      type: Opaque
9  stringData:
10  MYSQL_ROOT_PASSWORD: root
11  MYSQL_DATABASE: catalogue_db
12  MYSQL_USER: catalogue_user
13  MYSQL_PASSWORD: catalogue_pass

```

Listing 5.1 – Secret MySQL

### 5.3.2 ConfigMaps

```

1  apiVersion: v1
2  kind: ConfigMap
3  metadata:
4    name: emploi-app-config
5    namespace: planification
6    labels:
7      app: service-emploi-du-temps
8  data:
9    DATASOURCE_NAME: EmploiDS
10   DB_HOST: emploi-db
11   DB_PORT: "3306"

```



```
12 DB_NAME: emploi_db
13 DB_USER: emploi_user
14 CATALOGUE_BASE_URL: http://service-catalogue:8080
15 LOCAUX_BASE_URL: http://service-locaux:8080
```

Listing 5.2 – ConfigMap

Après le déploiement des ressources Kubernetes, il est important de vérifier la création des Secrets et des ConfigMaps. Les Secrets permettent de stocker les informations sensibles, telles que les identifiants de connexion aux bases de données, tandis que les ConfigMaps contiennent les paramètres de configuration des différents microservices.

La Figure 5.3 présente les Secrets et ConfigMaps créés dans le namespace `planification`. Ces ressources sont utilisées par les microservices lors de leur démarrage afin de charger automatiquement leur configuration.

```
vagrant@k3s-master:~/projet2-planification$ kubectl get secrets,configmaps -n planification
```

| NAME                       | TYPE   | DATA | AGE  |
|----------------------------|--------|------|------|
| secret/catalogue-db-secret | Opaque | 4    | 146m |
| secret/emploi-db-secret    | Opaque | 4    | 146m |
| secret/locaux-db-secret    | Opaque | 4    | 146m |

| NAME                                   | DATA | AGE  |
|----------------------------------------|------|------|
| configmap/catalogue-app-config         | 5    | 146m |
| configmap/catalogue-db-init-dtffc4959g | 1    | 146m |
| configmap/emploi-app-config            | 7    | 146m |
| configmap/kube-root-ca.crt             | 1    | 146m |
| configmap/locaux-app-config            | 5    | 146m |
| configmap/locaux-db-init-8h9fthgt4f    | 1    | 146m |

```
vagrant@k3s-master:~/projet2-planification$
```

FIGURE 5.3 – Secrets et ConfigMaps déployés dans le namespace `planification`

## 5.4 Persistance des données

Les trois bases MySQL sont déployées en **StatefulSet** [14] avec un service headless (`clusterIP: None`) pour une identité réseau stable. La persistance est assurée par des **volumeClaimTemplates** (pas de manifest PVC séparé) : chaque pod MySQL demande un volume **ReadWriteOnce** de 1 Gi. Les scripts `init.sql` de catalogue et locaux sont montés via ConfigMap (`catalogue-db-init`, `locaux-db-init`) générés par Kustomize. Les bases de données MySQL sont déployées sous forme de StatefulSets afin de garantir une identité stable pour chaque instance. Des Persistent Volume Claims (PVC) sont également créés afin d'assurer la persistance des données, même après le redémarrage des conteneurs.

La Figure 5.4 montre les StatefulSets ainsi que les volumes persistants associés aux différentes bases de données du projet.

```
vagrant@k3s-master:~/projet2-planification$
kubectl get statefulsets,pvc -n planification
NAME                                READY  AGE
statefulset.apps/catalogue-db      1/1    147m
statefulset.apps/emploi-db         1/1    147m
statefulset.apps/locaux-db         0/1    147m
```

| NAME                                              | STATUS | VOLUME                                   | CAPACITY | ACCESS MODES | STORAGECLASS | VOLUMEATTRIBUTESCL |
|---------------------------------------------------|--------|------------------------------------------|----------|--------------|--------------|--------------------|
| ASS AGE                                           |        |                                          |          |              |              |                    |
| persistentvolumeclaim/data-catalogue-db-0<br>147m | Bound  | pvc-25afb805-b1f0-4302-8f67-5f8b550645e6 | 1Gi      | RWO          | local-path   | <unset>            |
| persistentvolumeclaim/data-emploi-db-0<br>147m    | Bound  | pvc-100371e0-0c31-46c7-b7cb-54e56989d3a3 | 1Gi      | RWO          | local-path   | <unset>            |
| persistentvolumeclaim/data-locaux-db-0<br>147m    | Bound  | pvc-617c497e-d209-46a5-9ae4-19442ab16eb7 | 1Gi      | RWO          | local-path   | <unset>            |

```
vagrant@k3s-master:~/projet2-planification$
```

FIGURE 5.4 – StatefulSets et Persistent Volume Claims (PVC)

## 5.5 Exposition via Ingress Controller

```
1  apiVersion: networking.k8s.io/v1
2  kind: Ingress
3  metadata:
4  name: planification-ingress
5  namespace: planification
6  labels:
7  app.kubernetes.io/part-of: ump-planification
8  annotations:
9  # K3s utilise Traefik par défaut
10 traefik.ingress.kubernetes.io/router.entrypoints: web
11 spec:
12 ingressClassName: traefik
13 rules:
14 - host: planification.local
15 http:
16 paths:
17 # Catalogue -- CoursResource @Path("/cours")
18 - path: /api/cours
19 pathType: Prefix
20 backend:
21 service:
22 name: service-catalogue
23 port:
24 number: 8080
25 # Locaux -- LocalResource @Path("/locaux")
26 - path: /api/locaux
27 pathType: Prefix
28 backend:
29 service:
30 name: service-locaux
31 port:
32 number: 8080
33 # Emploi du temps -- EmploiDuTempsResource @Path("/emploi-du-temps")
34 - path: /api/emploi-du-temps
35 pathType: Prefix
36 backend:
37 service:
38 name: service-emploi-du-temps
39 port:
40 number: 8080
```

Listing 5.3 – Ingress – routage par chemin

En complément, des services NodePort (30081, 30082, 30083) permettent un accès direct sans Ingress. L'hôte `planification.local` doit être déclaré dans `/etc/hosts`. Afin de rendre les microservices accessibles depuis l'extérieur du cluster, un Ingress Controller Traefik [15] a été configuré. Celui-ci centralise les requêtes entrantes et les redirige vers le service approprié en fonction des règles définies.

La Figure 5.5 montre l'Ingress créé dans le namespace `planification`. Cette configuration permet d'accéder aux différents microservices à travers un point d'entrée unique.

```
vagrant@k3s-master:~/projet2-planification$ kubectl get ingress -n planification
NAME                CLASS    HOSTS                ADDRESS    PORTS    AGE
planification-ingress  traefik  planification.local  10.0.2.15  80      147m
vagrant@k3s-master:~/projet2-planification$
```

FIGURE 5.5 – Configuration de l'Ingress Kubernetes

# Chapitre 6

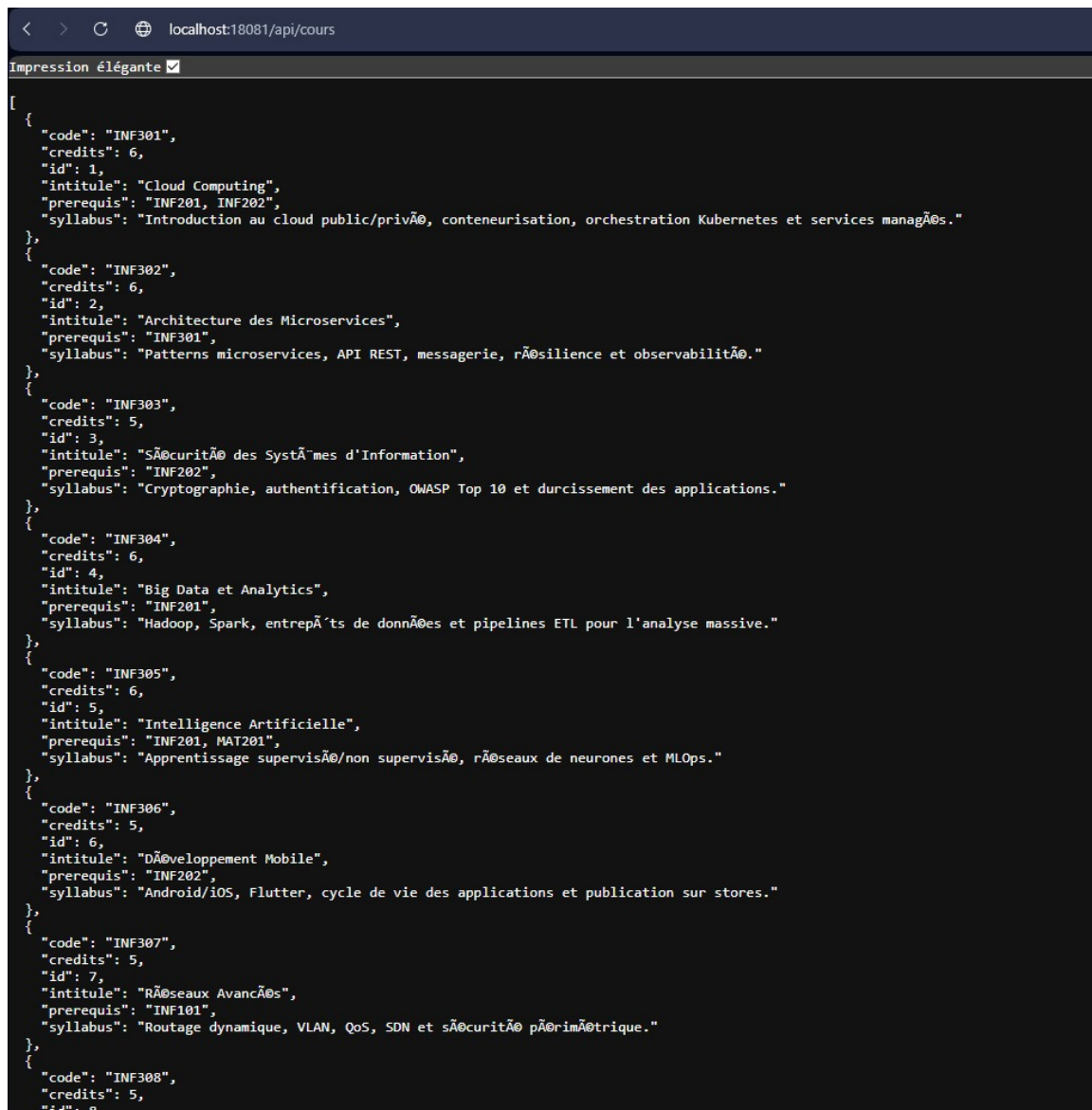
## Tests et validation

### 6.1 Tests fonctionnels

La validation repose sur le script `test.sh`, qui exécute 14 tests d'intégration automatisés sous Git Bash : healthchecks des trois services, création d'un cours et d'un local, planification d'une séance (HTTP 201), détection de conflit horaire (HTTP 409), consultation par jour, suppression (HTTP 204), vérification de la libération de salle (`disponibilite=DISPONIBLE`), puis tests des endpoints de lecture. Une collection Postman (`postman_collection.json`) et des guides de démo (`DEMO_TEST_*.txt`) documentent les scénarios manuels. Aucune capture d'écran n'est versionnée dans le dépôt ; les preuves sont produites à l'exécution (`docker compose ps`, sortie `[PASS]` de `test.sh`).

#### 6.1.1 Validation des API REST depuis le navigateur

Le premier test consiste à vérifier l'accès au microservice **Catalogue** depuis un navigateur web. En accédant à l'adresse `http://localhost:18081/api/cours`, le service retourne une réponse au format JSON contenant la liste des cours disponibles. Ce résultat confirme que l'API REST est accessible et que le service communique correctement avec sa base de données.



```

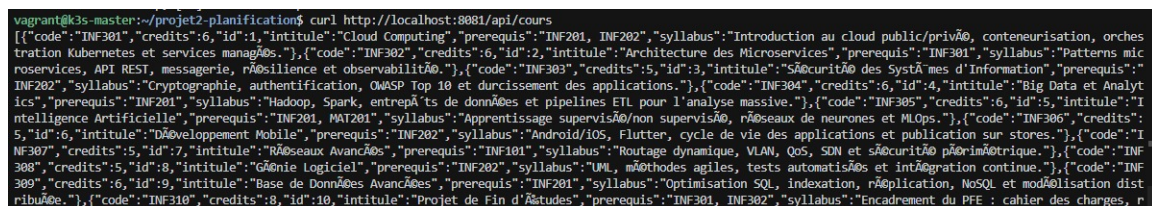
[
  {
    "code": "INF301",
    "credits": 6,
    "id": 1,
    "intitulé": "Cloud Computing",
    "prerequis": "INF201, INF202",
    "syllabus": "Introduction au cloud public/privé, conteneurisation, orchestration Kubernetes et services managés."
  },
  {
    "code": "INF302",
    "credits": 6,
    "id": 2,
    "intitulé": "Architecture des Microservices",
    "prerequis": "INF301",
    "syllabus": "Patterns microservices, API REST, messagerie, résilience et observabilité."
  },
  {
    "code": "INF303",
    "credits": 5,
    "id": 3,
    "intitulé": "Sécurité des Systèmes d'Information",
    "prerequis": "INF202",
    "syllabus": "Cryptographie, authentification, OWASP Top 10 et durcissement des applications."
  },
  {
    "code": "INF304",
    "credits": 6,
    "id": 4,
    "intitulé": "Big Data et Analytics",
    "prerequis": "INF201",
    "syllabus": "Hadoop, Spark, entrepôts de données et pipelines ETL pour l'analyse massive."
  },
  {
    "code": "INF305",
    "credits": 6,
    "id": 5,
    "intitulé": "Intelligence Artificielle",
    "prerequis": "INF201, MAT201",
    "syllabus": "Apprentissage supervisé/non supervisé, réseaux de neurones et MLOps."
  },
  {
    "code": "INF306",
    "credits": 5,
    "id": 6,
    "intitulé": "Développement Mobile",
    "prerequis": "INF202",
    "syllabus": "Android/iOS, Flutter, cycle de vie des applications et publication sur stores."
  },
  {
    "code": "INF307",
    "credits": 5,
    "id": 7,
    "intitulé": "Réseaux Avancés",
    "prerequis": "INF101",
    "syllabus": "Routing dynamique, VLAN, QoS, SDN et sécurité périmétrique."
  },
  {
    "code": "INF308",
    "credits": 5,
    "id": 8,
    "intitulé": "Génie Logiciel",
    "prerequis": "INF202",
    "syllabus": "UML, méthodes agiles, tests automatisés et intégration continue."
  },
  {
    "code": "INF309",
    "credits": 6,
    "id": 9,
    "intitulé": "Base de Données Avancées",
    "prerequis": "INF201",
    "syllabus": "Optimisation SQL, indexation, réplication, NoSQL et modélisation distribuée."
  },
  {
    "code": "INF310",
    "credits": 8,
    "id": 10,
    "intitulé": "Projet de Fin d'Études",
    "prerequis": "INF301, INF302",
    "syllabus": "Encadrement du PFE : cahier des charges, r

```

FIGURE 6.1 – Réponse JSON de l'API Catalogue affichée dans le navigateur

### 6.1.2 Validation des API REST avec la commande curl

Afin de confirmer le même comportement depuis un terminal, nous avons également testé l'API à l'aide de la commande `curl`. Cette commande permet d'envoyer une requête HTTP vers le microservice Catalogue et d'afficher directement la réponse JSON retournée.



```

vagrant@k3s-master:~/projet2-planification$ curl http://localhost:18081/api/cours
[{"code":"INF301","credits":6,"id":1,"intitulé":"Cloud Computing","prerequis":"INF201, INF202","syllabus":"Introduction au cloud public/privé, conteneurisation, orches
tration Kubernetes et services managés."}, {"code":"INF302","credits":6,"id":2,"intitulé":"Architecture des Microservices","prerequis":"INF301","syllabus":"Patterns mic
roservices, API REST, messagerie, résilience et observabilité."}, {"code":"INF303","credits":5,"id":3,"intitulé":"Sécurité des Systèmes d'Information","prerequis":"
INF202","syllabus":"Cryptographie, authentification, OWASP Top 10 et durcissement des applications."}, {"code":"INF304","credits":6,"id":4,"intitulé":"Big Data et Analyt
ics","prerequis":"INF201","syllabus":"Hadoop, Spark, entrepôts de données et pipelines ETL pour l'analyse massive."}, {"code":"INF305","credits":6,"id":5,"intitulé":"I
ntelligence Artificielle","prerequis":"INF201, MAT201","syllabus":"Apprentissage supervisé/non supervisé, réseaux de neurones et MLOps."}, {"code":"INF306","credits":
5,"id":6,"intitulé":"Développement Mobile","prerequis":"INF202","syllabus":"Android/iOS, Flutter, cycle de vie des applications et publication sur stores."}, {"code":"I
NF307","credits":5,"id":7,"intitulé":"Réseaux Avancés","prerequis":"INF101","syllabus":"Routing dynamique, VLAN, QoS, SDN et sécurité périmétrique."}, {"code":"INF
308","credits":5,"id":8,"intitulé":"Génie Logiciel","prerequis":"INF202","syllabus":"UML, méthodes agiles, tests automatisés et intégration continue."}, {"code":"INF
309","credits":6,"id":9,"intitulé":"Base de Données Avancées","prerequis":"INF201","syllabus":"Optimisation SQL, indexation, réplication, NoSQL et modélisation dist
ribuée."}, {"code":"INF310","credits":8,"id":10,"intitulé":"Projet de Fin d'Études","prerequis":"INF301, INF302","syllabus":"Encadrement du PFE : cahier des charges, r

```

FIGURE 6.2 – Test de l'API Catalogue avec la commande curl

## 6.2 Difficultés rencontrées et solutions

1. **Conteneurs WildFly Exited(255).** Cause : fins de ligne CRLF dans `entrypoint.sh`.  
Fix : `sed -i 's/\r$//'` dans chaque Dockerfile [3] et `.gitattributes (*.sh text eol=lf)`.
2. **HTTP 404 sur /api/cours.** Cause : context-root WildFly par défaut (/service-catalogue).  
Fix : `jboss-web.xml` avec `<context-root>/</context-root>`.
3. **Erreur JDBC / datasource absent.** Fix : module MySQL JDBC (`module.xml`) et configuration CLI `embed-server` dans `entrypoint.sh`.
4. **Conflit horaire retournait HTTP 500 au lieu de 409.** Cause : encapsulation EJB. Fix : `@ApplicationException(rollback=true)` sur `EmploiDuTempsException` et `EmploiDuTempsExceptionMapper`.
5. **POST cours en HTTP 500 (DataException).** Cause : code cours supérieur à 20 caractères. Fix : codes courts dans `test.sh` (`TST-{timestamp}`).
6. **curl JSON sous PowerShell.** Fix : utiliser Git Bash, Postman ou fichier JSON temporaire.
7. **Docker volumes sur vboxsf (Vagrant).** Fix : `rsync` vers ext4 natif dans `start-app.sh`.
8. **Timeout SSH Vagrant.** Fix : `boot_timeout = 600`, correctifs VirtualBox 7, réseau `host-only 192.168.56.x`.
9. **Ports 8081–8083 occupés.** Fix : forward Vagrant vers 18081–18083.
10. **Pod K3s ImagePullBackOff.** Fix : `import-images-k3s.sh`.
11. **MySQL K8s sans données initiales.** Fix : suppression des PVC puis redéploiement.
12. **Pod emploi-du-temps bloqué.** Fix : `initContainers wait-for-catalogue` et `wait-for-locaux`.
13. **Conteneurs non healthy avant 3 min.** Fix : `start_period: 60s` et attente 90–120 s au démarrage.

# Conclusion générale

Ce projet a permis de concevoir et de déployer une application de planification académique basée sur une architecture à microservices. Trois microservices développés avec Jakarta EE et WildFly ont été mis en œuvre afin de gérer les cours, les locaux et les emplois du temps. Chaque service dispose de sa propre base de données MySQL, garantissant ainsi une meilleure indépendance et une gestion distribuée des données.

L'application a été progressivement déployée en utilisant Docker Compose pour la conteneurisation, Vagrant pour la virtualisation de l'environnement et Kubernetes (K3s) pour l'orchestration des conteneurs. L'utilisation des StatefulSets, des Persistent Volume Claims, des Secrets, des ConfigMaps et de l'Ingress Controller a permis de mettre en place une architecture fiable, modulaire et facilement extensible.

Au-delà des aspects techniques, ce projet nous a permis d'acquérir une expérience pratique des principales technologies du Cloud Computing, notamment la conteneurisation, l'orchestration, le déploiement automatisé et l'administration d'une architecture distribuée.

Comme perspectives d'évolution, il serait intéressant d'ajouter une interface web complète, d'intégrer un registre privé d'images Docker, de mettre en œuvre un mécanisme de supervision et de monitoring, ainsi que de déployer l'application sur un cluster Kubernetes multi-nœuds afin d'améliorer sa tolérance aux pannes et sa capacité de montée en charge.



# Bibliographie

- [1] *About Oracle VirtualBox*. URL : <https://www.virtualbox.org/manual/UserManual.html> (visité le 05/07/2026).
- [2] *Compose file reference*. Docker Documentation. 100. URL : <https://docs.docker.com/reference/compose-file/> (visité le 05/07/2026).
- [3] *Dockerfile reference*. Docker Documentation. 200. URL : <https://docs.docker.com/reference/dockerfile/> (visité le 05/07/2026).
- [4] *Documentation - Hibernate ORM*. Hibernate. URL : <https://hibernate.org/orm/documentation/> (visité le 05/07/2026).
- [5] *Documentation | Vagrant | HashiCorp Developer*. Documentation | Vagrant | HashiCorp Developer. URL : <https://developer.hashicorp.com/vagrant/docs> (visité le 05/07/2026).
- [6] *Jakarta EE Platform 10 | Jakarta EE | The Eclipse Foundation*. Jakarta® EE : The New Home of Cloud Native Java. URL : <https://jakarta.ee/specifications/platform/10/> (visité le 05/07/2026).
- [7] *Jakarta Persistence | Jakarta EE | The Eclipse Foundation*. Jakarta® EE : The New Home of Cloud Native Java. URL : <https://jakarta.ee/specifications/persistence/> (visité le 05/07/2026).
- [8] *Jakarta RESTful Web Services | Jakarta EE | The Eclipse Foundation*. Jakarta® EE : The New Home of Cloud Native Java. URL : <https://jakarta.ee/specifications/restful-ws/> (visité le 05/07/2026).
- [9] *K3s - Lightweight Kubernetes | K3s*. 29 juin 2026. URL : <https://docs.k3s.io/> (visité le 05/07/2026).
- [10] *Kubernetes Documentation*. Kubernetes. URL : <https://kubernetes.io/docs/home/> (visité le 05/07/2026).
- [11] *Multi-stage builds*. Docker Documentation. 200. URL : <https://docs.docker.com/build/building/multi-stage/> (visité le 05/07/2026).
- [12] *MySQL : : MySQL 8.0 Reference Manual*. URL : <https://dev.mysql.com/doc/refman/8.0/en/> (visité le 05/07/2026).
- [13] *Secrets*. Kubernetes. Section : docs. URL : <https://kubernetes.io/docs/concepts/configuration/secret/> (visité le 05/07/2026).
- [14] *StatefulSets*. Kubernetes. Section : docs. URL : <https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/> (visité le 05/07/2026).
- [15] *Traefik Kubernetes Ingress Documentation - Traefik*. URL : <https://doc.traefik.io/traefik/reference/install-configuration/providers/kubernetes/kubernetes-ingress/> (visité le 05/07/2026).



- 
- [16] *WildFly Documentation*. URL : <https://docs.wildfly.org/27/> (visit   le 05/07/2026).